

Lecture 10: Data Visualization (Tidyverse)

Harris Coding Camp – Standard Track

Summer 2024

Data Visualization

- ▶ Be able to plot in `ggplot`
- ▶ Know what types of figures you want
- ▶ Use plots to gain information
 - ▶ graphs, including axes, must be labeled properly to allow readers to understand the visualization with ease
 - ▶ variable scales should be portrayed accurately
 - ▶ graphs should be as intuitive as possible

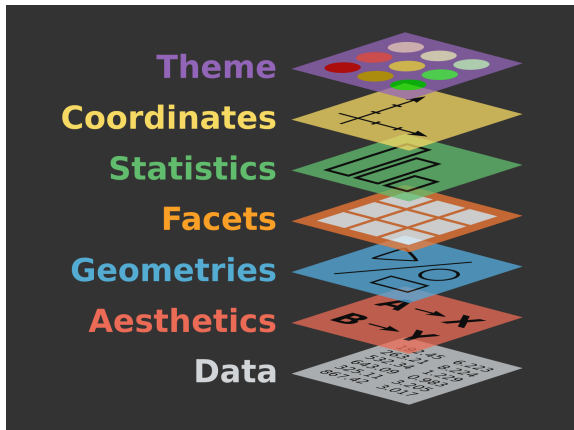
What is ggplot?

- ▶ ggplot is a data visualization package
- ▶ It is part of the tidyverse suite of packages

```
# need to load tidyverse first  
library(tidyverse)
```

Get started with ggplot

- ▶ How to map data to aesthetics with `aes`
- ▶ How to visualize the mappings with `geoms`
- ▶ How to get more out of your data by using multiple aesthetics
- ▶ How to use `facets` to add dimensionality



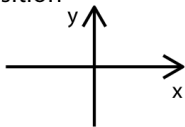
Basic Components of ggplot (Layers)

- ▶ Layer 1: Background layer (`ggplot()`)
 - ▶ A data frame
 - ▶ Aesthetic mapping: how data are mapped to x-axis, y-axis, color, size, etc
- ▶ Layer 2: Geometry layer (`geom_xxx()`)
 - ▶ geometric objects like points, lines, shapes
- ▶ Layer 3: Labels Layer (`labs()`)
 - ▶ title, legend, etc
- ▶ And more!

What are Aesthetic Mappings?

- ▶ Describe how variables are mapped to visual properties or aesthetics
 - ▶ In tidyverse, aesthetic means “something you can see”
 - ▶ Including things like size, shape, color or x and y variables
- ▶ To display values, map variables in the data to visual properties of the geom

position



shape



size



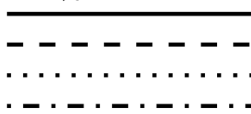
color



line width



line type



Simplest ggplot code structure

```
ggplot(data = [dataset],  
       mapping = aes(x = [x-variable],  
                     y = [y-variable]) +  
                     ... )  
  
geom_<name>() +  
other options
```

ggplot(): Layer 1, data frame

Let's get started! ggplot() tells R to prepare to make a plot.

```
# Let's prepare new data frame 'annual_sales'  
# First, sum up monthly volume to get annual volume  
annual_sales <-  
  housing_data %>%  
    group_by(year) %>%  
    summarize(total_volume = sum(volume, na.rm = TRUE))
```

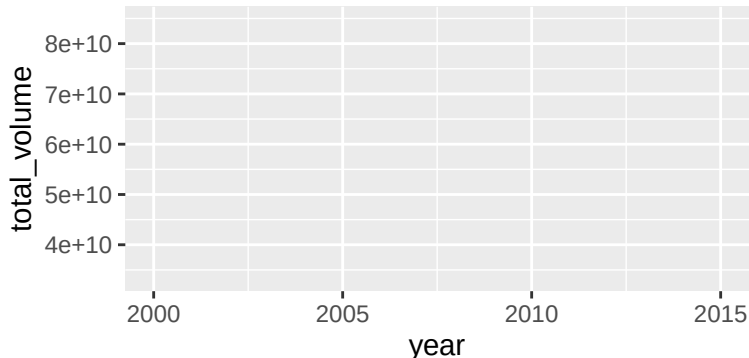
```
# Layer 1, data frame  
ggplot(data = annual_sales)
```


ggplot(): Layer 1, adding a mapping

Adding mapping = aes() says how the data will map to “aesthetics”:

- ▶ e.g. tell R to make x-axis year and y-axis total_volume

```
ggplot(data = annual_sales,  
       mapping = aes(x = year, y = total_volume))
```

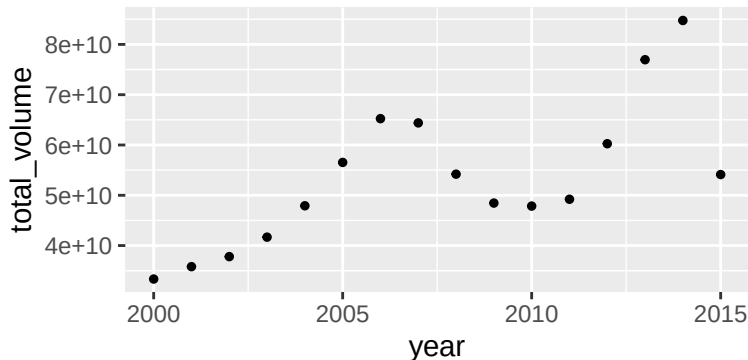


ggplot(): Layer 2, visualizing the mapping with geom

`geom_<name>` tells R what type of visualization to produce.

Here we see **points** by using `geom_point()`:

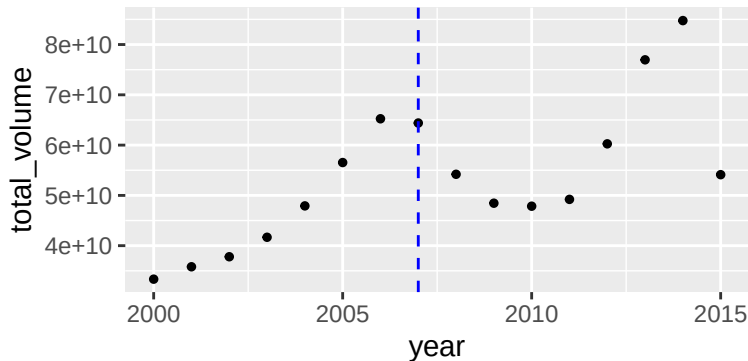
```
ggplot(data = annual_sales,  
       mapping = aes(x = year, y = total_volume)) +  
  geom_point(color = "black", size = 1, shape = 19)
```



ggplot(): Layer 2, visualizing the mapping with geom

- Often, we may include more info by adding a **vertical line** to a plot:

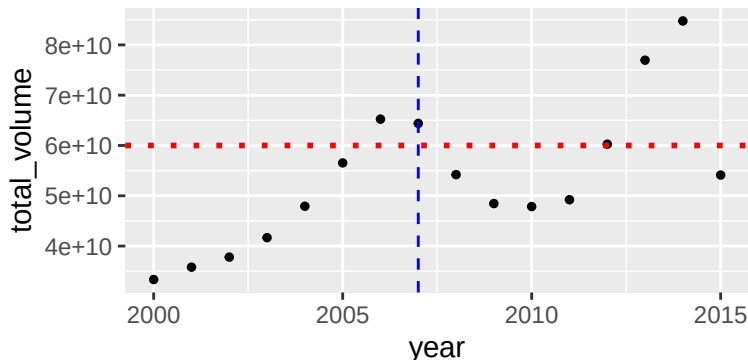
```
annual_sales %>%  
  ggplot(aes(x = year, y = total_volume)) +  
    geom_point(color = "black", size = 1) +  
    geom_vline(xintercept = 2007, linetype = "dashed",  
              color = "blue", size = 0.5)
```



ggplot(): Layer 2, visualizing the mapping with geom

- ...and by adding a **horizontal line** to a plot:

```
annual_sales %>%  
  ggplot(aes(x = year, y = total_volume)) +  
    geom_point(color = "black", size = 1) +  
    geom_vline(xintercept = 2007, linetype = "dashed",  
              color = "blue", size = 0.5) +  
    geom_hline(yintercept = 6e10, linetype = "dotted",  
              color = "red", size = 1)
```

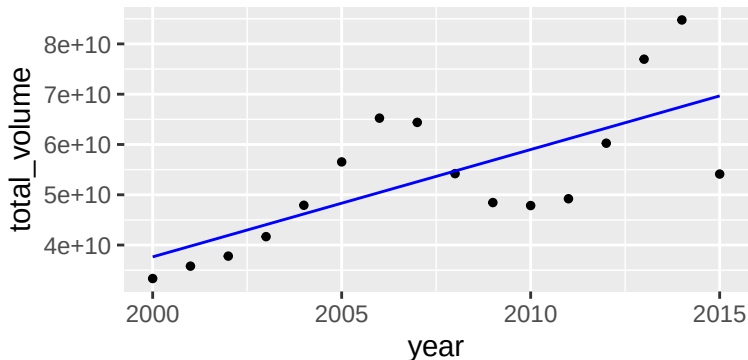


ggplot(): Layer 2, visualizing the mapping with geom

- ▶ We can impose the regression line using `geom_smooth`
- ▶ `lm` means linear model; `se = FALSE` disables confidence interval

```
annual_sales %>%  
  ggplot(aes(x = year, y = total_volume)) +  
    geom_point(color = "black", size = 1) +  
    geom_smooth(method = "lm", se = FALSE, color = "blue", size = 0.5)
```

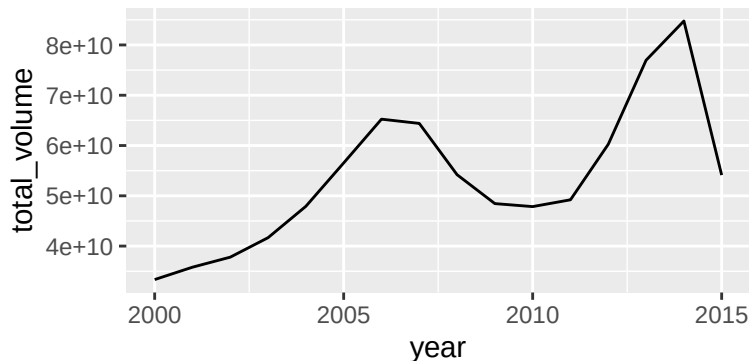
```
## `geom_smooth()` using formula = 'y ~ x'
```



ggplot(): Layer 2, visualizing the mapping with geom

We can create a line plot by using `geom_line()`.

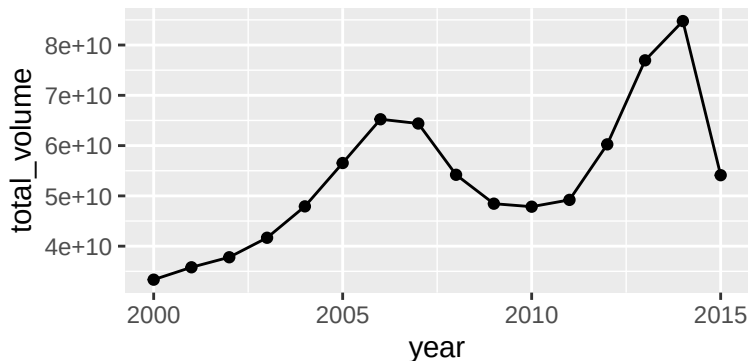
```
ggplot(data = annual_sales,  
       mapping = aes(x = year, y = total_volume)) +  
  geom_line()
```



ggplot(): Layer 2, visualizing the mapping with geom

The data can be visualized with different geoms that can be composed (+) together:

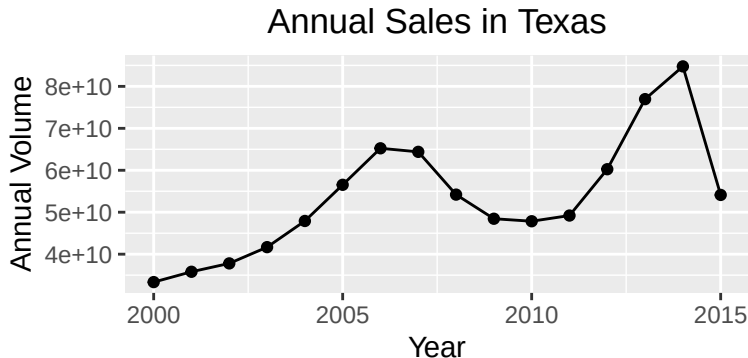
```
ggplot(data = annual_sales,  
       mapping = aes(x = year, y = total_volume)) +  
  geom_line() +  
  geom_point()
```



ggplot(): Layer 3, adding labels

Adding labels makes the plot more readable:

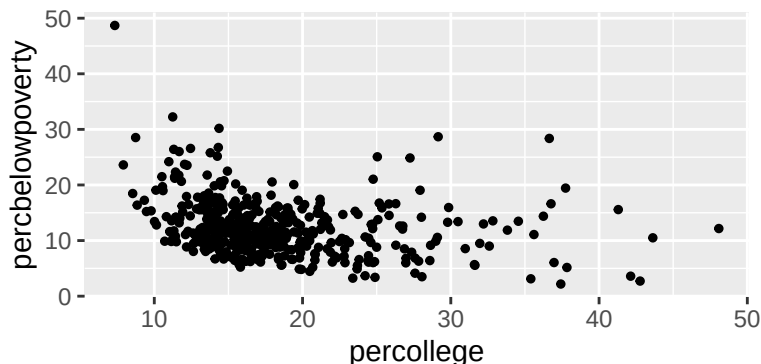
```
ggplot(data = annual_sales,  
       mapping = aes(x = year, y = total_volume)) +  
  geom_line() +  
  geom_point() +  
  labs(x = "Year", y = "Annual Volume") +  
  ggtitle("Annual Sales in Texas") +  
  theme(plot.title = element_text(hjust = 0.5)) #center the title
```



ggplot(): More on aesthetics

Let's use midwest data and start with only mapping to x and y

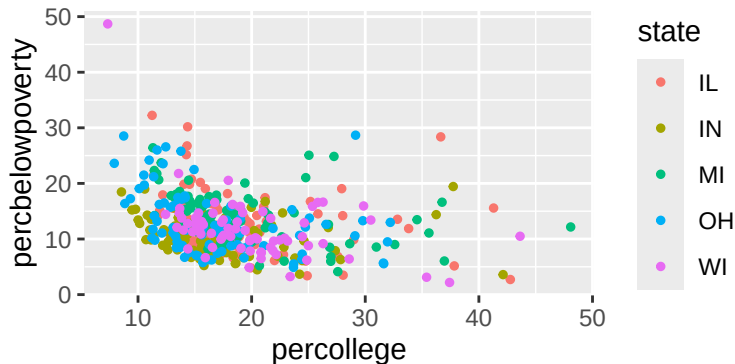
```
midwest %>%  
  ggplot(aes(x = percollege, y = percbelowpoverty)) +  
  geom_point(size = 1)
```



ggplot(): Using color

- ▶ color maps data to the color of points
 - ▶ Each state is assigned a color

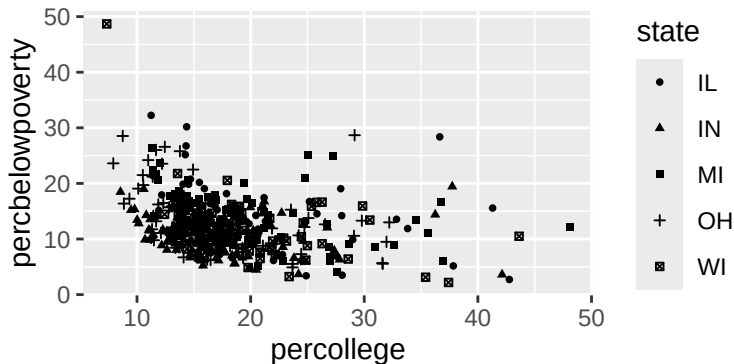
```
midwest %>%  
  ggplot(aes(x = percollege, y = percbelowpoverty,  
             color = state)) +  
  geom_point(size = 1)
```



ggplot(): Using shape

- ▶ shape maps data to the shape of points
 - ▶ Each state is assigned a shape

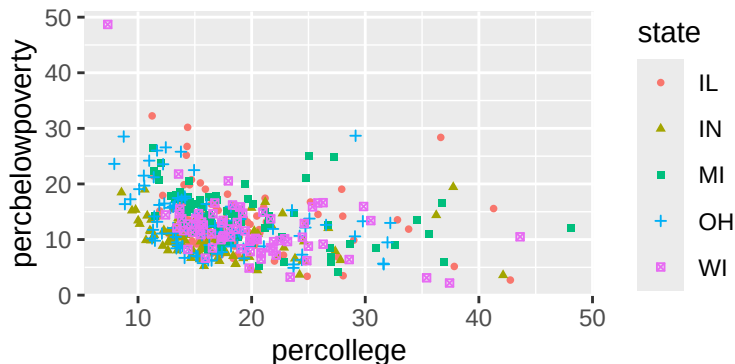
```
midwest %>%  
  ggplot(aes(x = percollege, y = percbelowpoverty,  
             shape = state)) +  
  geom_point(size = 1)
```



ggplot(): Using color + shape

- ▶ Combining color and shape:
 - ▶ Each state is assigned a shape and color

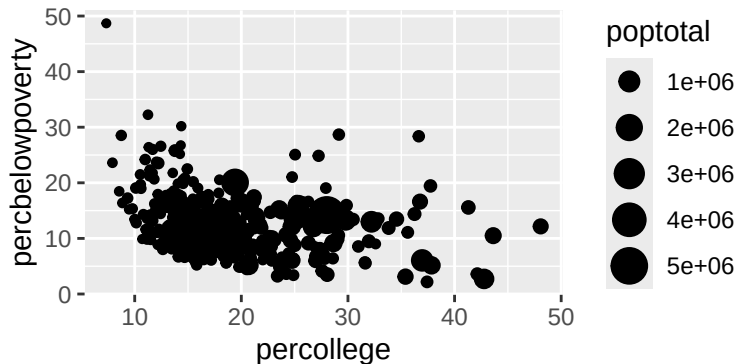
```
midwest %>%  
  ggplot(aes(x = percollege, y = perbelowpoverty,  
             color = state,  
             shape = state)) +  
  geom_point(size = 1)
```



ggplot(): Using size

- ▶ size maps data to the size of points
- ▶ we map the percentage of people within a known poverty status to size

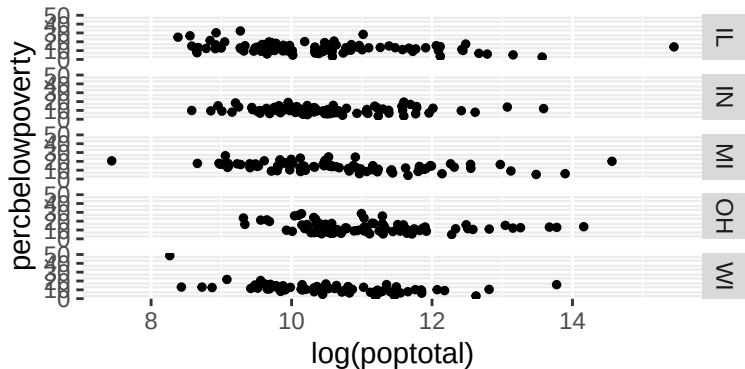
```
midwest %>%  
  ggplot(aes(x = percollege, y = percbelowpoverty,  
             size = poptotal)) +  
  geom_point()
```



ggplot(): Facets (facet_grid)

Facets provide an additional tool to explore multi-dimensional data:

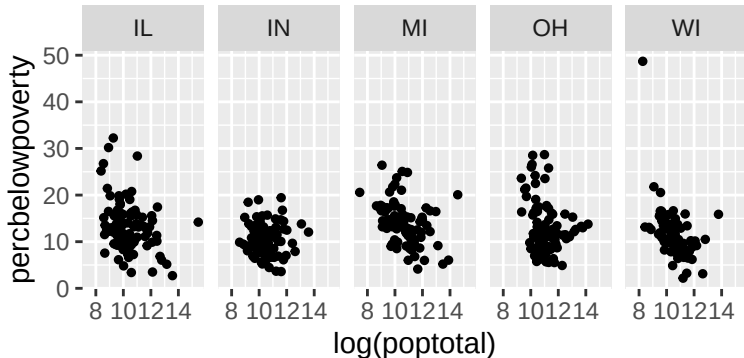
```
midwest %>%  
  ggplot(aes(x = log(poptotal), y = percbelowpoverty)) +  
    geom_point(size = 1) +  
    facet_grid(state ~ .) # Faceting by 'state' along rows
```



ggplot(): Facets (facet_grid)

Facets provide an additional tool to explore multi-dimensional data:

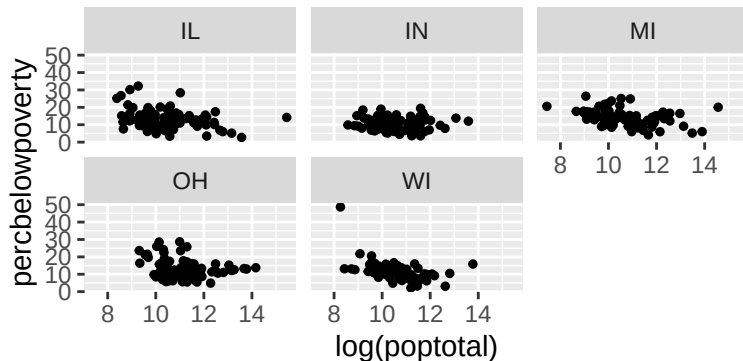
```
midwest %>%  
  ggplot(aes(x = log(poptotal), y = percbelowpoverty)) +  
    geom_point(size = 1) +  
    facet_grid(. ~ state) # Faceting by 'state' along columns
```



ggplot(): Facets (facet_wrap)

Facets provide an additional tool to explore multi-dimensional data:

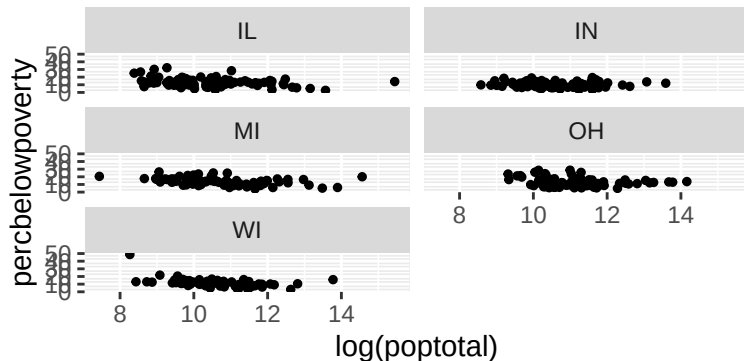
```
midwest %>%  
  ggplot(aes(x = log(poptotal), y = percbelowpoverty)) +  
    geom_point(size = 1) +  
    facet_wrap(~ state, nrow = 2, ncol = 3) # 2 rows and 3 columns
```



ggplot(): Facets (facet_wrap)

Facets provide an additional tool to explore multi-dimensional data:

```
midwest %>%  
  ggplot(aes(x = log(poptotal), y = percbelowpoverty)) +  
    geom_point(size = 1) +  
    facet_wrap(~ state, nrow = 3, ncol = 2) # 3 rows and 2 columns
```



ggplot(): Using aesthetics to explore data

The ggplot cheatsheet shows all the geoms with their associated aesthetics

Data visualization with ggplot2 : : CHEAT SHEET



Basics

ggplot2 is based on the **grammar of graphics**, the idea that you can build every graph from the same components: a **data** set, a **coordinate system**, and **geoms**—visual marks that represent data points.



To display values, map variables in the data to visual properties of the geom (**aesthetics**) like **size**, **color**, and **x** and **y** locations.



Complete the template below to build a graph.

```
ggplot(data = <DATA>) +  
  <GEOM FUNCTION> (mapping = aes (<MAPPINGS>))  
  stat = <STAT>, position = <POSITION> +  
  <COORDINATE FUNCTION> +  
  <FACET FUNCTION> +  
  <SCALE FUNCTION> +  
  <THEME FUNCTION>
```

Not required, accessible defaults supplied

`ggplot(data = mpg, aes(x = cty, y = hwy))` Begins a plot that you finish by adding layers to. Add one geom function per layer.

`last_plot()` Returns the last plot.

`ggsave("plot.png", width = 5, height = 5)` Saves last plot as 5" x 5" file named "plot.png" in working directory. Matches file type to file extension.

Geoms

Use a **geom** function to represent data points, use the **geom's** aesthetic properties to represent variables. Each function returns a layer.

GRAPHICAL PRIMITIVES

`a <- ggplot(economics, aes(date, unemployment))`
`b <- ggplot(seals, aes(x = long, y = lat))`

- a + geom_blank()** and **a + expand_limits()**
Ensure limits include values across all plots.
- b + geom_curve()** `aes(yend = lat + 1, xend = long + 1, curvature = 1) - x, yend, y, yend, alpha, angle, color, curvature, linetype, size`
- a + geom_path()** `lineend = "butt", linejoin = "round", linewidth = 1`
`x, y, alpha, color, group, linetype, size`
- a + geom_polygon()** `aes(alpha = 50)` - `x, y, alpha, color, fill, group, linetype, size`
- b + geom_rect()** `aes(xmin = long, ymin = lat, xmax = long + 1, ymax = lat + 1)` - `xmax, xmin, ymax, ymin, alpha, color, fill, linetype, size`
- a + geom_ribbon()** `aes(ymin = unemployment - 900, ymax = unemployment + 900)` - `x, ymax, ymin, alpha, color, fill, group, linetype, size`

LINE SEGMENTS

common aesthetics: `x, y, alpha, color, linetype, size`

- b + geom_abline()** `aes(intercept = 0, slope = 1)`
- b + geom_hline()** `aes(yintercept = lat)`
- b + geom_vline()** `aes(xintercept = long)`

- b + geom_segment()** `aes(yend = lat + 1, xend = long + 1)`
- b + geom_spoke()** `aes(angle = 1:1155, radius = 1)`

ONE VARIABLE continuous

`c <- ggplot(mpg, aes(hwy)); c2 <- ggplot(mpg)`

- c + geom_area()** `stat = "bin"`
`x, y, alpha, color, fill, linetype, size`
- c + geom_density()** `kernel = "gaussian"`
`x, y, alpha, color, fill, group, linetype, size, weight`

TWO VARIABLES

both continuous
`e <- ggplot(mpg, aes(cty, hwy))`

- e + geom_label()** `aes(label = cty, nudges_x = 1, nudges_y = 1) - x, y, label, alpha, angle, color, family, fontface, fsize, linewidth, size, vjust`
- e + geom_point()**
`x, y, alpha, color, fill, shape, size, stroke`
- e + geom_quantile()**
`x, y, alpha, color, group, linetype, size, weight`
- e + geom_rug()** `sides = "bl"`
`x, y, alpha, color, linetype, size`
- e + geom_smooth()** `method = lm`
`x, y, alpha, color, fill, group, linetype, size, weight`
- e + geom_text()** `aes(label = cty, nudges_x = 1, nudges_y = 1) - x, y, label, alpha, angle, color, family, fontface, fsize, linewidth, size, vjust`

one discrete, one continuous

`f <- ggplot(mpg, aes(class, hwy))`

- f + geom_col()**
`x, y, alpha, color, fill, group, linetype, size`
- f + geom_boxplot()**
`x, y, lower, middle, upper, ymax, ymin, alpha, color, fill, group, linetype, shape, size, weight`
- f + geom_dotplot()** `binaxis = "y", stackdir = "center"`
`x, y, alpha, color, fill, group`
- f + geom_violin()** `scale = "area"`
`x, y, alpha, color, fill, group, linetype, size, weight`

both discrete

continuous bivariate distribution

`h <- ggplot(diamonds, aes(carat, price))`

- h + geom_bin2d()** `binwidth = c(0.25, 500)`
`x, y, alpha, color, fill, linetype, size, weight`
- h + geom_density_2d()**
`x, y, alpha, color, group, linetype, size`
- h + geom_hex()**
`x, y, alpha, color, fill, size`

continuous function

`i <- ggplot(economics, aes(date, unemployment))`

- i + geom_area()**
`x, y, alpha, color, fill, linetype, size`
- i + geom_line()**
`x, y, alpha, color, group, linetype, size`
- i + geom_step()** `direction = "hv"`
`x, y, alpha, color, group, linetype, size`

visualizing error

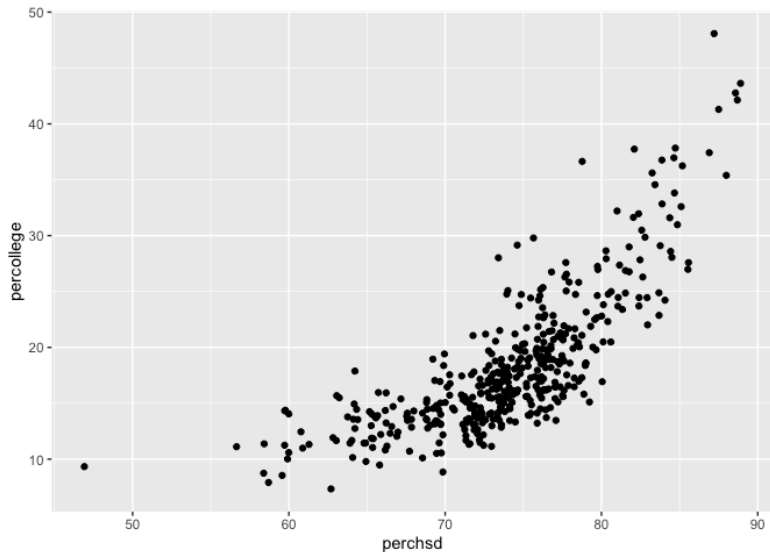
`df <- data.frame(grp = c("A", "B"), fit = 4.5, se = 1.2)`
`j <- ggplot(df, aes(grp, fit, ymin = fit - se, ymax = fit + se))`

- j + geom_crossbar()** `latten = 2` - `x, y, ymax, ymin, alpha, color, fill, group, linetype, size`
- j + geom_errorbar()** - `x, ymax, ymin, alpha, color, group, linetype, size, width`
Also `geom_errorbarh()`.
- j + geom_linerange()**
`x, ymin, ymax, alpha, color, group, linetype, size`
- j + geom_pointrange()** - `x, y, ymin, ymax, alpha, color, fill, group, linetype, shape, size`

maps

Try it yourself: Plot 1

1. Adjust code to reproduce the following plot (sample codes provided in the next slide):

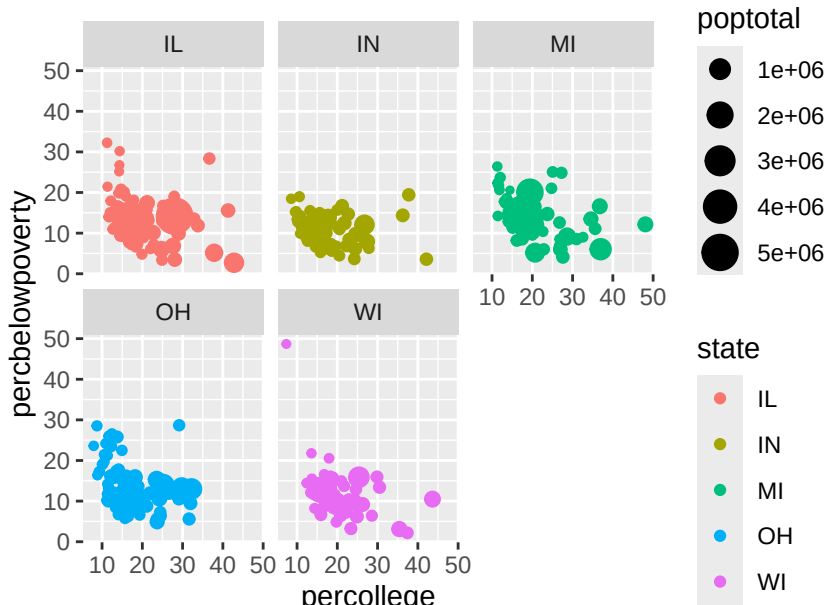


Try it yourself: Plot 1

```
midwest %>%  
  ggplot(aes(x = ?,  
             y = ?)) +  
  geom_?()
```

Try it yourself: Plot 2

2. Adjust code to reproduce the following plot (sample codes provided):



Try it yourself: Plot 2

```
midwest %>%  
  ggplot(aes(x = ?,  
             y = ?,  
             color = ?,  
             size = ?)) +  
  geom_?() +  
  facet_?(vars(?))
```

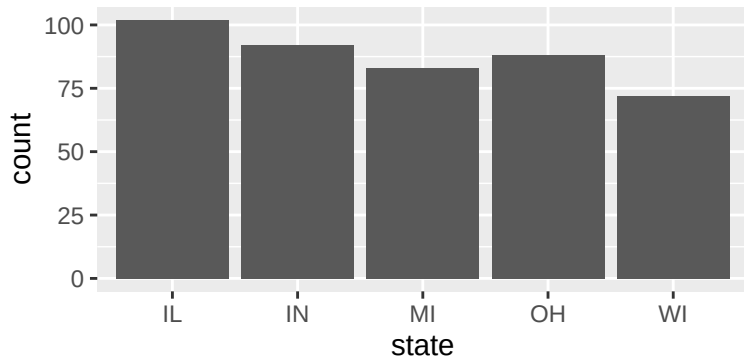
Type of figures

Type of figures

1. Distribution of **univariate (single variable)**
 - ▶ bar plot, histogram, density plot, etc
2. Relationship between **bivariate (two variables)**
 - ▶ scatter plot, line plot, boxplot, (segmented) bar plot, etc
3. Relationship between **many variables** at once
 - ▶ usually focusing on the relationship between two while conditioning on others

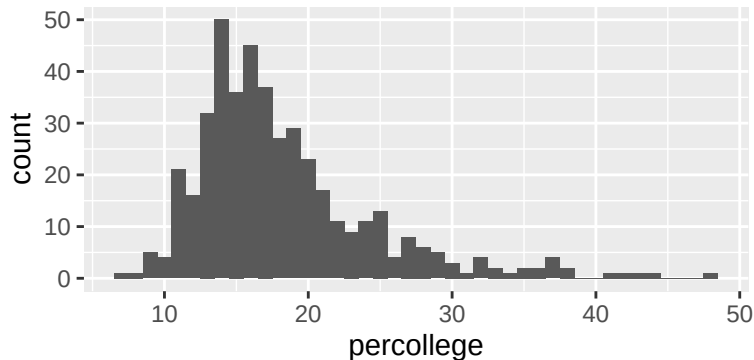
Univariate: bar plot

```
midwest %>%  
  ggplot(aes(x = state)) +  
    geom_bar()
```



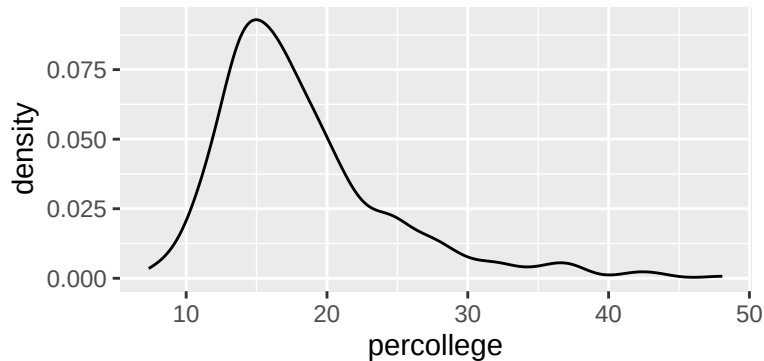
Univariate: histogram

```
midwest %>%  
  ggplot(aes(x = percollege)) +  
    geom_histogram(binwidth = 1)
```



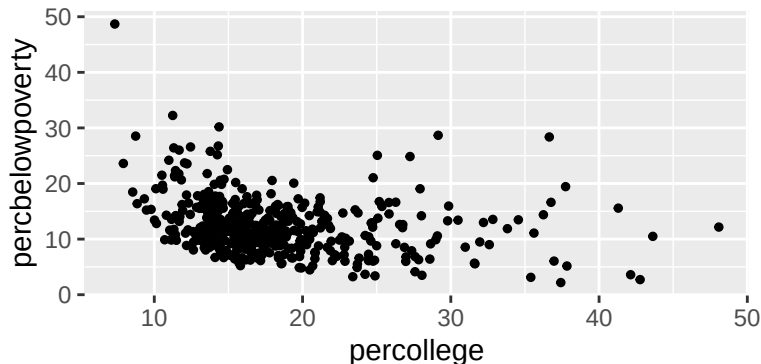
Univariate: density

```
midwest %>%  
  ggplot(aes(x = percollege)) +  
    geom_density()
```



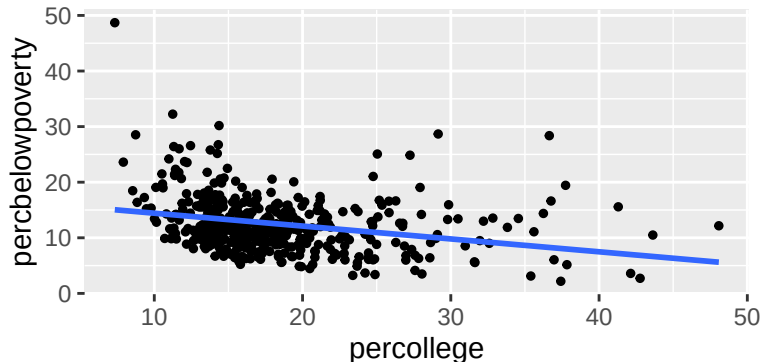
Bivariate: scatter plot

```
midwest %>%  
  ggplot(aes(x = percollege,  
             y = percbelowpoverty)) +  
  geom_point(size = 1)
```



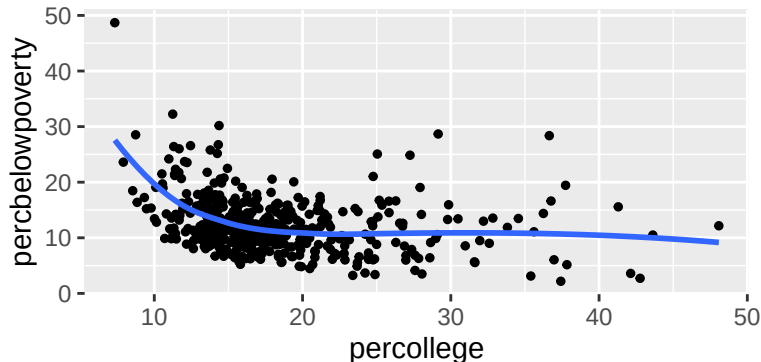
Bivariate: scatter + linear line plot

```
midwest %>%  
  ggplot(aes(x = percollege,  
             y = percbelowpoverty)) +  
  geom_point(size = 1) +  
  geom_smooth(method = "lm", se = FALSE)
```



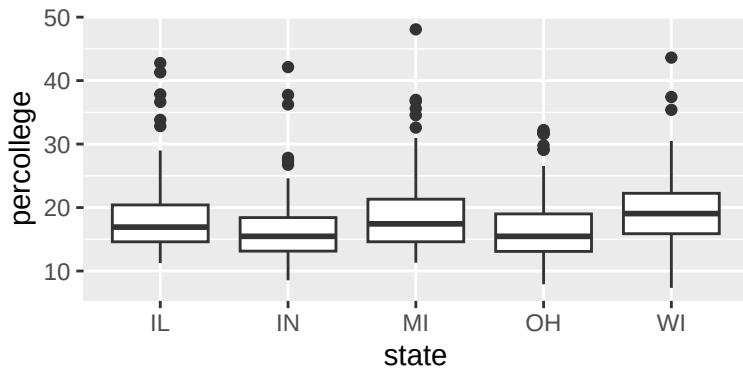
Bivariate: scatter + smooth line plot

```
midwest %>%  
  ggplot(aes(x = percollege,  
             y = percbelowpoverty)) +  
  geom_point(size = 1) +  
  geom_smooth(se = FALSE)
```



Bivariate: box plots

```
midwest %>%  
  ggplot(aes(x = state,  
             y = percollege)) +  
  geom_boxplot()
```



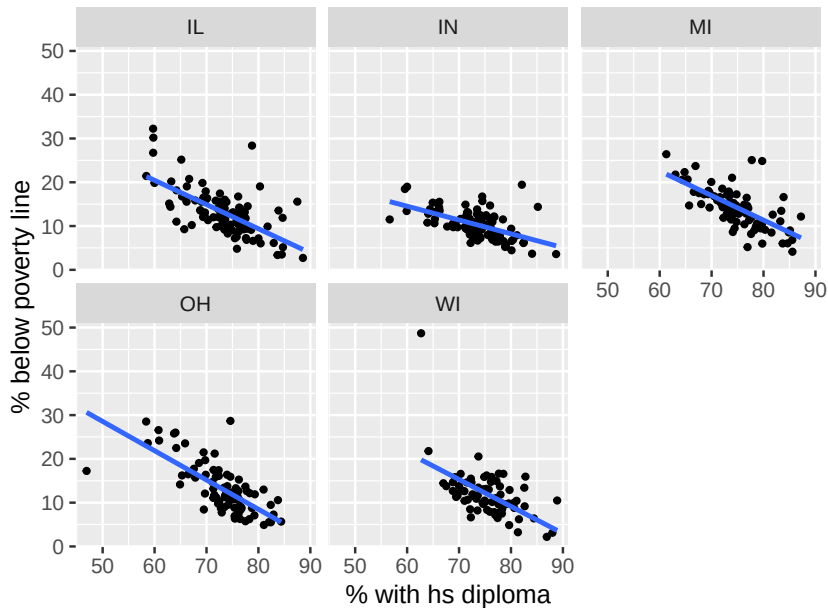
Try it yourself: Plot 3

3. Adjust code to reproduce the plot in the next slide:

```
midwest %>%  
  ggplot(aes(x = perchsd,  
             y = percbelowpoverty)) +  
  geom_?(size = 1) +  
  geom_?(method = "?", se = F) +  
  facet_?(vars(?)) +  
  labs(x = "?", y = "?") +  
  ggtitle("?") +  
  theme(plot.title = element_text(hjust = 0.5)) #center the title
```


Try it yourself: Plot 3

Degree vs Poverty Rate



Recap

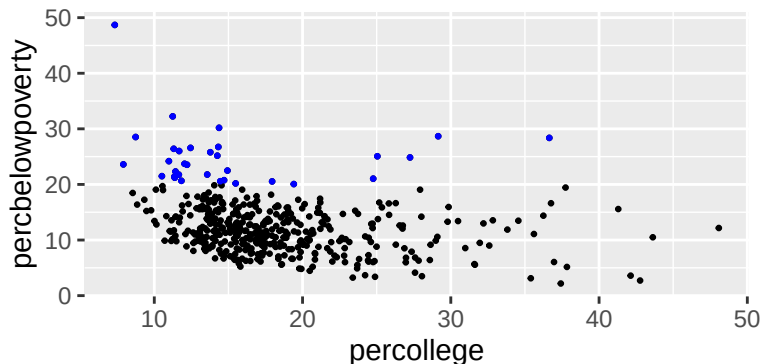
- ▶ Visualizing our data can help lead to powerful insights between variable relationships
 - ▶ Making quick plots helps us understand data and makes us aware of data issues
- ▶ `ggplot` starts by mapping data to “aesthetics”
 - ▶ e.g. What data shows up on x and y axes and how color, size and shape appear on the plot
- ▶ Then, we use `geoms` to create a visualization based on the mapping
- ▶ We many consider adding labels to make plots more readable
- ▶ There are many ways you can visualize your data!

Appendix

ggplot(): Using color

- To highlight points based on a certain threshold in a plot, use `geom_point(data = your_dataset %>% filter(your_condition))`

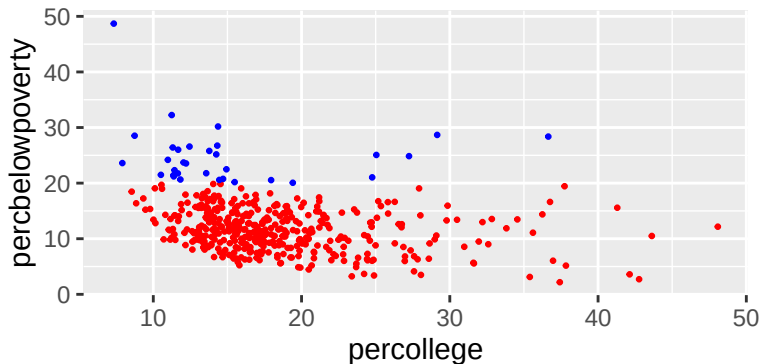
```
midwest %>%  
  ggplot(aes(x = percollege, y = percbelowpoverty)) +  
    geom_point(size = 0.5) +  
    geom_point(data = midwest %>%  
      filter(percbelowpoverty >= 20), color = "blue", size =
```



ggplot(): Using color

- To highlight points based on a certain threshold in a plot, use `geom_point(data = your_dataset %>% filter(your_condition))`

```
midwest %>%  
  ggplot(aes(x = percollege, y = percbelowpoverty)) +  
    geom_point(data = midwest %>%  
      filter(percbelowpoverty < 20), color = "red", size = 0.  
    geom_point(data = midwest %>%  
      filter(percbelowpoverty >= 20), color = "blue", size =
```



ggplot(): Using color

- To highlight points below the regression line:

```
# Fit a linear regression model
reg <- lm(total_volume ~ year, data = annual_sales)

# Get the fitted values (predicted values)
annual_sales$predicted <- predict(reg)

# Add a logical column to check if points are below the line
annual_sales$below_line <-
  annual_sales$total_volume < annual_sales$predicted

# Plot it!
annual_sales %>%
  ggplot(aes(x = year, y = total_volume)) +
  geom_point(aes(color = below_line)) +
  geom_smooth(method = "lm", se = FALSE, color = "blue", size = 0.5)
```

Appendix: Some graphs you made along the way

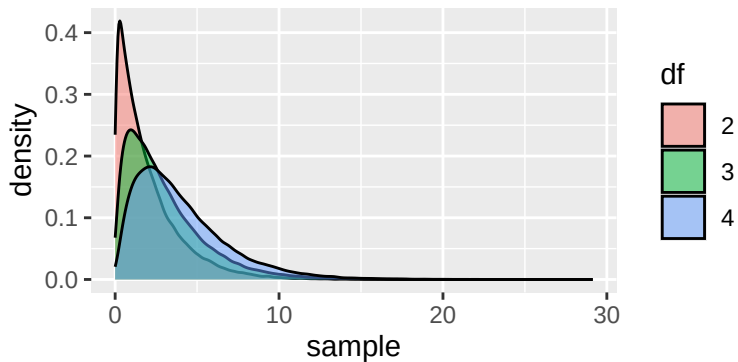
- ▶ Distributions
- ▶ Grouped bar graph
- ▶ Faceted bar graph

Appendix: distributions

- ▶ `geom_density()` only requires an `x` aesthetic and it calculates the distribution to plot.
- ▶ We can set the aesthetics manually, independent of data for nicer graphs.

```
chi_sq_samples <-  
  tibble(x = c(rchisq(100000, 2),  
               rchisq(100000, 3),  
               rchisq(100000, 4)),  
         df = rep(c("2", "3", "4"), each = 1e5))  
  
chi_sq_samples %>%  
  ggplot(aes(x = x, fill = df)) +  
  geom_density(alpha = .5) +  
  labs(fill = "df", x = "sample")
```


Appendix: distributions

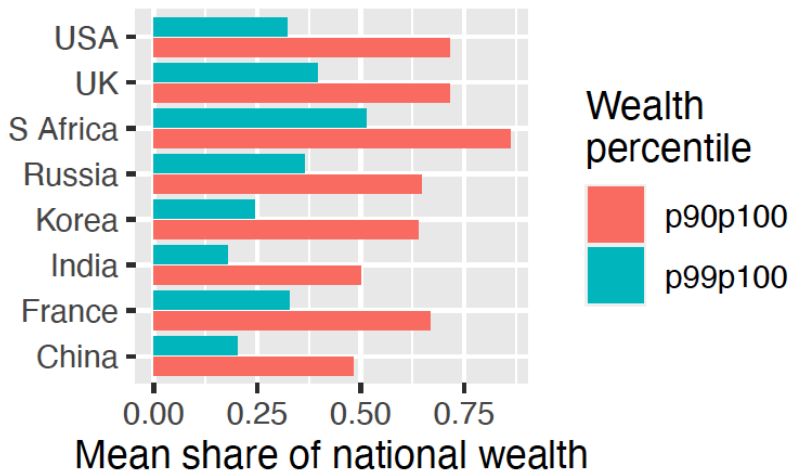


Appendix: grouped bar graph

- ▶ `position = "dodge2"` tells R to put bars next to each other, rather than stacked on top of each other.
- ▶ We use `fill` and not `color` because we're "filling" an area.

```
mean_share_per_country %>%  
  ggplot(aes(y = country,  
             x = mean_share,  
             fill = percentile)) +  
  geom_col(position = "dodge2") +  
  labs(x = "Mean share of national wealth",  
       y = "",  
       fill = "Wealth\npercentile")
```

Appendix: grouped bar graph



Appendix: faceted bar graph

- ▶ Notice that we manipulate our data to the right specification before making this graph
- ▶ Using `facet_wrap` we get a distinct graph for each time period.

```
mean_share_per_country_with_time %>%  
  ggplot(aes(x = country,  
             y = mean_share,  
             fill = percentile)) +  
  geom_col(position = "dodge2") +  
  facet_wrap(vars(time_period))
```

Appendix: faceted bar graph

